

JavaScript Interview Questions & Answers

20 questions, progressing from basic syntax to core concepts to technical depth

Basic Syntax

1. What are the different ways to declare a variable in JavaScript, and how do they differ?

'var' is function-scoped, gets hoisted and initialized as undefined, and can be redeclared. 'let' is block-scoped, hoisted but not initialized (temporal dead zone), and can be reassigned but not redeclared in the same scope. 'const' is also block-scoped, but cannot be reassigned after initialization — note this only prevents reassignment, not mutation of an object's contents.

2. What are the primitive data types in JavaScript?

There are seven primitive types: string, number, boolean, undefined, null, symbol, and bigint. Everything else (objects, arrays, functions) is an object type, which is stored and compared by reference rather than by value.

3. What is the difference between '==' and '===' in JavaScript?

'==' (loose equality) compares two values after converting them to a common type if they differ, which can produce surprising results (e.g. '0 == false' is true). '===' (strict equality) compares both value and type with no conversion, and is generally the recommended default to avoid unexpected coercion bugs.

4. What is the difference between 'null' and 'undefined'?

'undefined' means a variable has been declared but not yet assigned a value, or a function has no explicit return value. 'null' is an intentional assignment representing 'no value' or 'empty', explicitly set by a developer. `typeof undefined` is 'undefined', while `typeof null` is (famously, and somewhat incorrectly) 'object'.

5. How do template literals differ from regular strings?

Template literals, written with backticks, allow embedded expressions using `${...}` syntax (string interpolation) and support multi-line strings without needing explicit newline characters, unlike single or double-quoted strings which require concatenation for both.

Common & Important Concepts

1. What is the difference between a function declaration and a function expression?

A function declaration (`function foo() {}`) is hoisted entirely, so it can be called before its definition in the code. A function expression (`const foo = function() {}`) is only hoisted as a variable, not with its function body, so calling it before the assignment throws an error.

2. What are arrow functions, and how do they differ from regular functions?

Arrow functions (`() => {}`) offer shorter syntax and, importantly, do not have their own 'this' binding — they inherit 'this' lexically from the enclosing scope. They also lack their own 'arguments' object and cannot be used as constructors with 'new'.

3. What is the difference between 'map()', 'forEach()', and 'filter()' on arrays?

'forEach()' iterates over an array and executes a callback for each item, returning undefined — used purely for side effects. 'map()' also iterates, but returns a new array containing the callback's return value for each item. 'filter()' returns a new array containing only the items for which the callback returns true.

4. What is destructuring, and can you give an example?

Destructuring lets you unpack values from arrays or properties from objects into distinct variables in a concise syntax. For example: `const { name, age } = person;` extracts 'name' and 'age' directly from the 'person' object, and `const [first, second] = arr;` does the same for array elements by position.

5. What is the spread operator, and how is it different from the rest parameter?

Both use '...' syntax but serve opposite purposes. The spread operator expands an iterable into individual elements — e.g. `[...arr1, ...arr2]` merges arrays, or `{...obj1, ...obj2}` merges objects. The rest parameter does the opposite, collecting multiple arguments or remaining elements into a single array — e.g. `function sum(...nums) {}` gathers all arguments into the 'nums' array.

Technical Deep-Dive

1. Explain closures in JavaScript with an example of a practical use case.

A closure is formed when a function retains access to variables from its enclosing lexical scope even after that outer function has finished executing. A common practical use is creating private state: `function createCounter() { let count = 0; return () => ++count; }` — the returned function keeps its own private 'count' that persists across calls and can't be accessed directly from outside.

2. What is the event loop, and how does it relate to the call stack, task queue, and microtask queue?

JavaScript is single-threaded, executing synchronous code on the call stack. Asynchronous callbacks (like `setTimeout`) go into the task (macrotask) queue, while Promise callbacks go into the microtask queue. The event loop continuously checks: once the call stack is empty, it processes all pending microtasks first, then takes one task from the macrotask queue — which is why Promises typically resolve before `setTimeout` callbacks even with a 0ms delay.

3. What is 'this' in JavaScript, and how is its value determined?

'this' refers to the context in which a function is called, and its value is determined at call time based on how the function is invoked, not where it's defined (except for arrow functions, which use lexical 'this'). In a plain function call it's undefined (strict mode) or the global object; as a method call, it's the object before the dot; with `call`/`apply`/`bind`, it's explicitly set; and with 'new', it's the newly created object.

4. What is prototypal inheritance, and how does it differ from classical inheritance?

JavaScript objects inherit directly from other objects via a prototype chain: when a property isn't found on an object, the engine looks up its `[[Prototype]]` link, continuing until it's found or the chain ends at null. Classical inheritance (as in Java or C++) instead defines inheritance through classes as blueprints. ES6 'class' syntax in JavaScript is syntactic sugar over this same prototypal mechanism.

5. What are Promises, and how do 'async/await' relate to them?

A Promise represents the eventual result of an asynchronous operation, existing in one of three states: pending, fulfilled, or rejected, and is consumed via `.then()/.catch()`. 'async/await' is syntactic sugar built on top of Promises: an 'async' function always returns a Promise, and 'await' pauses execution within that function until the awaited Promise settles, letting asynchronous code read in a more synchronous, linear style.